

Тема 7 Команды модификации языка DML. Введение в представления

Ввод значений

Значения могут быть помещены и удалены из полей таблиц БД тремя командами языка DML (Язык Манипулирования Данными):

**INSERT (ВСТАВИТЬ),
UPDATE (МОДИФИЦИРОВАТЬ),
DELETE (УДАЛИТЬ).**

Все строки в SQL вводятся с использованием команды модификации INSERT. В самой простой форме, INSERT использует следующий синтаксис:

**INSERT INTO <table name>
VALUES (<value>, <value> . . .);**

Так, например, чтобы ввести строку в таблицу Продавцов, вы можете использовать следующее условие:

**INSERT INTO Salespeople
VALUES (1001, 'Peel', 'London', .12);**

Команды DML не производят никакого вывода, но программа должна дать вам некоторое подтверждение того, что данные были использованы.

Имя таблицы (в нашем случае – Salespeople – Продавцы) должно быть предварительно определено в команде CREATE TABLE, а каждое значение, пронумерованное в предложении значений, должно совпадать с типом данных столбца, в который оно вставляется. В ANSI, эти значения не могут составлять выражений, что означает, что 3 – это доступно, а выражение 2 + 1 – нет. Значения, конечно же, вводятся в таблицу в поименном порядке, поэтому первое значение с именем, автоматически попадает в столбец 1, второе в столбец 2, и так далее.

Если нужно ввести пустое значение (NULL), вы вводите его точно также как и обычное значение. Предположим, что еще не имелось поля city для Peel. Вы можете вставить его строку со значением равным NULL в это поле, следующим образом:

**INSERT INTO Salespeople
VALUES (1001, 'Peel', NULL, .12);**

Так как значение NULL – это специальный маркер, а не просто символьное значение, оно не заключается в одиночные кавычки.

Можно также указывать столбцы, куда вы хотите вставить значение имени. Это позволяет вам вставлять имена в любом Заказе. Предположим, что вы берете значения для таблицы Заказчиков из отчета, выводимого на принтер, который помещает их в таком Заказе: city, cname, и snum, и для упрощения, вы хотите ввести значения в том же Заказе:

**INSERT INTO Customers (city, cname, cnum)
VALUES ('London', 'Honman', 2001);**

Обратите внимание, что столбцы rating и snum – отсутствуют. Это значит, что эти строки автоматически установлены в значение по умолчанию. По умолчанию может быть введено или значение NULL или другое значение, определяемое как значение по умолчанию. Если ограничение запрещает использование значения NULL в данном столбце, и этот столбец не установлен как по умолчанию, этот столбец должен быть обеспечен значением для любой команды INSERT, которая относится к таблице.

Вы можете также использовать команду INSERT, чтобы получать или выбирать значения из одной таблицы и помещать их в другую, чтобы использовать их вместе с запросом. Чтобы сделать это, надо заменить предложение VALUES (из предыдущего примера) на соответствующий запрос:

```
INSERT INTO Londonstaff  
SELECT *  
FROM Salespeople  
WHERE city = 'London';
```

Здесь выбираются все значения, произведенные запросом – то есть, все строки из таблицы Продавцов со значениями city = "London" помещаются в таблицу называемую Londonstaff. Чтобы это работало, таблица Londonstaff должна отвечать следующим условиям:

- * Она должна уже быть создана командой CREATE TABLE.

- * Она должна иметь четыре столбца, которые совпадают с таблицей Продавцов в терминах типа данных; то есть первый, второй, и так далее, столбцы каждой таблицы должны иметь одинаковый тип данных (причем они не обязаны иметь одинаковых имен).

Общее правило то, что вставляемые столбцы таблицы, должны совпадать со столбцами выводимыми подзапросом, в данном случае, для всей таблицы Продавцов.

Londonstaff – это теперь независимая таблица, которая получила некоторые значения из таблицы Продавцов (Salespeople). Если значения в таблице Продавцов будут вдруг изменены, это никак не отразится на таблице Londonstaff.

Так как или запрос или команда INSERT могут указывать столбцы по имени, вы можете переместить только выбранные столбцы, а также переупорядочить только те столбцы, которые вы выбрали.

Например, надо сформировать новую таблицу с именем Daytotals, которая будет следить за общим количеством сумм приобретений упорядоченных на каждый день. Вы можете ввести эти данные независимо от таблицы Заказов, но сначала вы должны заполнить таблицу Daytotals информацией ранее представленной в таблице Заказов.

Понимая, что таблица Заказов охватывает последний финансовый год, а не только несколько дней, как в нашем примере, вы можете видеть преимущество использования следующего условия INSERT в подсчете и вводе значений

```
INSERT INTO Daytotals (date, total)  
SELECT odate, SUM (amt)  
FROM Orders  
GROUP BY odate;
```

Обратите внимание что, как предложено ранее, имена столбцов таблицы Заказов и таблицы Daytotals – не должны быть одинаковыми. Кроме того, если дата приобретения и общее количество – это единственные столбцы в таблице, и они находятся в данном Заказе, их имена могут быть исключены из вывода из-за их очевидной простоты.

Удаление строк

Можно удалять строки из таблицы командой модификации – DELETE. Она удаляет только введенные строки, а не индивидуальные значения полей, так что параметр поля является необязательным или недоступным. Чтобы удалить все содержание таблицы Продавцов, вы можете ввести следующее:

```
DELETE FROM Salespeople;
```

Теперь, когда таблица пуста, ее можно окончательно удалить командой DROP TABLE.

Обычно, нужно удалить только некоторые определенные строки из таблицы. Чтобы определить какие строки будут удалены, вы используете предикат, так же как мы это делали для запросов. Например, чтобы удалить продавца Axelrod из таблицы, можно ввести:

```
DELETE FROM Salespeople  
WHERE snum = 1003;
```

Мы использовали поле snum вместо поля sname потому, что это лучшая тактика при использовании первичных ключей, когда вы хотите чтобы действию подвергалась одна и только одна строка.

Конечно, можно также использовать DELETE с предикатом, который бы выбирал группу строк, как показано в этом примере:

```
DELETE FROM Salespeople  
WHERE city = 'London';
```

Изменение значений поля

Изменение некоторых или всех значений в существующей строке выполняется командой UPDATE.

Эта команда содержит предложение UPDATE, в котором указано имя используемой таблицы, и предложение SET, указывающее на изменение, которое нужно сделать для определенного столбца. Например, чтобы изменить оценки всех заказчиков на 200, вы можете ввести

```
UPDATE Customers  
SET rating = 200;
```

Конечно, вы не всегда захотите указывать все строки таблицы для изменения единственного значения, так что UPDATE, аналогично DELETE, может брать предикаты. Например, можно выполнить изменение, одинаковое для всех заказчиков продавца Peel (имеющего snum=1001):

```
UPDATE Customers  
SET rating = 200  
WHERE snum = 1001;
```

Предложение SET может назначать любое число столбцов, отделяемых запятыми. Все указанные назначения могут быть сделаны для любой табличной строки, но только для одной в каждый момент времени. Предположим, что продавец Motika ушел на пенсию, и мы хотим переназначить его номер новому продавцу:

```
UPDATE Salespeople  
SET sname = 'Gibson', city = 'Boston', comm = .10  
WHERE snum = 1004;
```

Эта команда передаст новому продавцу Gibson, всех текущих заказчиков бывшего продавца Motika и Заказы, в том виде, в котором они были скомпонованы для Motika.

Нельзя, однако, модифицировать сразу много таблиц в одной команде, частично потому, что вы не можете использовать префиксы таблицы со столбцами измененными предложением SET. Другими словами, вы не можете сказать – "SET Salespeople.sname = Gibson" в команде UPDATE, вы можете сказать только так – "SET sname = Gibson".

Можно использовать скалярные выражения в предложении SET команды UPDATE, включив его в выражение поля, которое будет изменено. В этом их отличие от предложения VALUES команды INSERT, в котором выражения не могут использоваться. Например, надо удвоить комиссионные всем вашим продавцам. Это можно сделать с следующим виде:

```
UPDATE Salespeople  
SET comm = comm * 2;
```

Всякий раз, когда надо сослаться к указанному значению столбца в предложении SET, произведенное значение может получиться из текущей строки, прежде в ней будут сделаны какие-то изменения с помощью команды UPDATE. Можно скомбинировать эти особенности, и сказать, – удвоить комиссию всем продавцам в Лондоне, таким способом:

```
UPDATE Salespeople  
SET comm = comm * 2  
WHERE city = 'London';
```

Предложение SET – это не предикат и позволяет вводить пустые NULL значения. Так что, если надо установить все оценки заказчиков в Лондоне в NULL, вы можете ввести следующее:

```
UPDATE customers  
SET rating = NULL  
WHERE city = 'London';
```

что обнулит все оценки заказчиков в Лондоне.

Использование подзапросов с INSERT

Можно использовать подзапросы внутри любого запроса, который генерирует значения для команды INSERT тем же самым способом, которым вы делали это для других запросов, т.е. внутри предиката или предложения HAVING.

Предположим, что мы имеем таблицу с именем SJpeople, столбцы которой совпадают со столбцами таблицы Продавцов. Например, надо заполнять таблицу заказчиками в городе San Jose:

```
INSERT INTO SJpeople  
SELECT *  
FROM Salespeople  
WHERE city = 'San Jose';
```

Теперь мы можем использовать подзапрос, например, чтобы добавить к таблице SJpeople всех продавцов, которые имеют заказчиков в San Jose, независимо от того, находятся ли там продавцы или нет:

```
INSERT INTO Sjpeople  
SELECT *  
FROM Salespeople  
WHERE snum = ANY (SELECT snum  
                  FROM Customers  
                  WHERE city = 'San Jose');
```

Оба запроса в этой команде функционируют так же, как если бы они не являлись частью выражения INSERT. Подзапрос находит все строки для заказчиков в San Jose и формирует набор значений snum. Внешний запрос выбирает строки из таблицы Salespeople, где эти значения snum найдены.

Запрещение на ссылку к таблице, которая модифицируется командой INSERT, не предохранит вас от использования подзапросов, которые ссылаются к таблице, используемой в предложении FROM внешней команды SELECT. Таблица, из которой выбираются значения, чтобы произвести их для INSERT, не будет задействована командой; и можно будет ссылаться к этой таблице любым способом, которым обычно это делали, но только если эта таблица указана в автономном запросе. Например, что имеется таблица с именем Samecity, в которой мы запоним продавцов с заказчиками в их городах.

Мы можем заполнить таблицу, используя соотнесенный подзапрос:

```
INSERT INTO (Samecity  
SELECT *  
FROM (Salespeople outer  
WHERE city IN (SELECT city  
              FROM Customers inner  
              WHERE inner.snum = outer.snum));
```

Ни таблица Samecity, ни таблица Продавцов не должны быть использованы во внешних или внутренних запросах INSERT.

В качестве другого примера, предположим, что имеется премия для продавца, который имеет самый большой заказ на каждый день. Следить за этим в таблице с именем Bonus, которая содержит поле snum продавцов, поле odate и поле amt. Надо заполнить эту таблицу информацией, которая хранится в таблице Заказов, используя следующую команду:

```
INSERT INTO Bonus  
SELECT snum, odate, amt  
FROM Orders a  
WHERE amt = (SELECT MAX (amt)  
                FROM Orders b  
                WHERE a.odate = b.odate);
```

Даже если эта команда имеет подзапрос, который базируется на той же самой таблице, что и внешний запрос, он не ссылается к таблице Bonus, на которую воздействует команда. Что для нас абсолютно приемлемо.

Логика запроса должна просматривать таблицу Заказов, и находить для каждой строки максимум Заказа сумм приобретений для этой даты. Если эта величина такая же, как у текущей строки – текущая строка является наибольшим Заказом для этой даты, и данные вставляются в таблицу Bonus.

Использование подзапросов с DELETE

Можно также использовать подзапросы в предикате команды DELETE. Это даст возможность определять некоторые довольно сложные критерии, чтобы установить, какие строки будут удаляться.

Например, чтобы удалить всех заказчиков, назначенных к продавцам в Лондоне, то можно использовать следующий запрос,:

```
DELETE  
FROM Customers  
WHERE snum = ANY (SELECT snum  
                    FROM Salespeople  
                    WHERE city = 'London');
```

Нельзя ссылаться к таблице, из которой вы будете удалять строки, в предложении FROM подзапроса, вы можете в предикате сослаться на текущую строку-кандидат этой таблицы, которая является строкой, которая в настоящее время проверяется в основном предикате. Другими словами, вы можете использовать соотнесенные подзапросы. Они отличаются от тех соотнесенных подзапросов, которые вы могли использовать с INSERT, в котором они фактически базировались на строках-кандидатах таблицы, задействованной в команде, а не на запросе другой таблицы.

```
DELETE FROM Salespeople  
WHERE EXISTS (SELECT *  
              FROM Customers  
              WHERE rating = 100 AND  
                Salespeople.snum = Customers.snum);
```

Обратите внимание, что AND часть предиката внутреннего запроса ссылается к таблице Продавцов. Это означает, что весь подзапрос будет выполняться отдельно для каждой строки таблицы Продавцов, также как это выполнялось с другими соотнесенными подзапросами. Эта команда удалит всех продавцов которые имели, по меньшей мере, одного заказчика с оценкой 100 в таблице Продавцов.

Имеется другой способ сделать то же:

```
DELETE FROM Salespeople  
WHERE 100 IN (SELECT rating  
              FROM Customers  
              WHERE Salespeople.snum = Customers.snum);
```

Эта команда находит все оценки для каждого заказчика продавцов и удаляет тех продавцов, заказчики которых имеют оценку rating = 100.

Обычно соотнесенные подзапросы – это подзапросы, связанные с таблицей, к которой они ссылаются во внешнем запросе (а не в самом предложении DELETE) – и также часто используемы. Можно найти наименьший заказ на каждый день и удалить продавцов, которые произвели его, с помощью следующей команды:

```
DELETE FROM Salespeople
WHERE (snum IN (SELECT snum
FROM Orders a
WHERE amt = (SELECT MIN (amt)
FROM Orders b
WHERE a.odate = b.odate));
```

Подзапрос в предикате DELETE берет соотнесенный подзапрос. Этот внутренний запрос находит минимальную сумму заказа для даты каждой строки внешнего запроса. Если эта сумма такая же, как сумма текущей строки, предикат внешнего запроса верен, что означает, что текущая строка имеет наименьший заказ для этой даты. Поле snum продавца, ответственного за этот заказ, извлекается и передается в основной предикат команды DELETE, которая затем удаляет все строки с этим значением поля snum из таблицы Продавцов. Так как snum это первичный ключ таблицы Продавцов, то, естественно там должна иметься только одна удаляемая строка для значения поля snum выведенного с помощью подзапроса. Если имеется больше одной строки, все они будут удалены.

Использование подзапросов с UPDATE

UPDATE использует подзапросы тем же самым способом, что и DELETE. Например, с помощью соотнесенного подзапроса к таблице, которая будет модифицироваться, вы можете увеличить комиссионные всех продавцов которые были назначены по крайней мере двум заказчикам:

```
UPDATE Salespeople
SET comm = comm + .01
WHERE 2 <= (SELECT COUNT (cnum)
FROM Customers
WHERE Customers.snum = Salespeople.snum);
```

Теперь продавцы Peel и Serres, имеющие многочисленных заказчиков, получают повышение своих комиссионных.

Другой пример уменьшает комиссионные продавцов, которые произвели наименьшие Заказы, но не стирает их в таблице:

```
UPDATE salespeople
SET comm = comm - .01
WHERE snum IN (SELECT snum
FROM Orders a
WHERE amt = (SELECT MIN (amt)
FROM Orders b
WHERE a.odate = b.odate));
```

Например, вы не можете просто выполнить такую операцию, как удаление всех заказчиков с оценками ниже средней. Вероятно, лучше всего можно бы сначала (Шаг 1.), выполнить запрос, получающий среднюю величину, а затем (Шаг 2.), удалить все строки с оценкой ниже этой величины:

Шаг 1.

```
SELECT AVG (rating)
FROM Customers;
```

Вывод = 200.

Шаг 2.

```
DELETE
FROM Customers
WHERE rating < 200;
```


Что такое представление?

Типы таблиц, с которыми мы имели дело до сих пор, назывались – *базовыми таблицами*. Это – таблицы, которые содержат данные. Однако имеется другой вид таблиц – представления. *Представления* – это таблицы, содержание которых выбирается или получается из других таблиц.

Представления подобны окнам, через которые просматривается информация, которая фактически хранится в базовой таблице. *Представление* – это фактически запрос, который выполняется всякий раз, когда представление становится темой команды. Вывод запроса при этом в каждый момент становится содержанием представления.

Представление создаётся командой CREATE VIEW. Она состоит из слов CREATE VIEW (СОЗДАТЬ ПРЕДСТАВЛЕНИЕ), имени представления, которое нужно создать, слова AS (КАК), и далее запроса, как в следующем примере:

```
CREATE VIEW Londonstaff  
AS SELECT *  
FROM Salespeople  
WHERE city = 'London';
```

Теперь создано представление, называемое Londonstaff. Это представление можно использовать так же, как и любую другую таблицу. Она может быть запрошена, модифицирована, вставлена в., удалена из..., и соединена с., другими таблицами и представлениями.

Когда вы хотите выбрать (SELECT) все строки (*) из представления, он выполняет запрос, содержащийся в определении Londonstaff, и возвращает все из его вывода.

```
Select *  
FROM Londonstaff;
```

Имея предикат в запросе представления, можно вывести только те строки из представления, которые будут удовлетворять этому предикату. Преимущество использования представления, по сравнению с использованием основной таблицы в том, что представление будет модифицировано автоматически всякий раз, когда таблица, лежащая в его основе – изменяется.

Содержание представления не фиксировано, и переназначается каждый раз, когда вы ссылаетесь на представление в команде. Если вы добавите другого, живущего в Лондоне продавца, он автоматически появится в представлении.

Представления значительно расширяют управление данными таблиц. Это хороший способ дать публичный доступ к некоторой, но не всей информации в таблице. Если вы хотите, чтобы продавец был показан в таблице Продавцов, но при этом не были показаны комиссии других продавцов, можно создать представление с использованием следующей команды:

```
CREATE VIEW Salesown  
AS SELECT snum, sname, city  
FROM Salespeople;
```

Другими словами, это представление – такое же, как и таблиц Продавцов, за исключением того, что поле comm не упоминалось в запросе, и, следовательно, не было включено в представление.

Представление может изменяться командами модификации DML, но модификация не будет воздействовать на само представление. Команды будут на самом деле перенаправлены к базовой таблице:

```
UPDATE Salesown  
SET city = 'Palo Alto'  
WHERE snum = 1004;
```

Это действие идентично выполнению той же команды в таблице Продавцов. Однако, если значение комиссионных продавца будет обработано командой UPDATE

```
UPDATE Salesown  
SET comm = .20  
WHERE snum = 1004;
```

она будет отвергнута, так как поле comm отсутствует в представлении Salesown. Это важное замечание, показывающее, что не все представления могут быть модифицированы.

В нашем примере, поля представлений имеют свои имена, полученные прямо из имен полей основной таблицы. Это удобно. Однако, иногда нужно, чтобы столбцы имели новые именами:

- когда некоторые столбцы являются выводимыми и поэтому не имеющими имен;
- когда два или более столбцов в объединении имеют те же имена, что в их базовой таблице.

Имена, которые могут стать именами полей, даются в круглых скобках после имени таблиц. Они не будут запрошены, если совпадают с именами полей запрашиваемой таблицы. Тип данных и размер этих полей будут отличаться от типа данных и размера запрашиваемых полей, которые "передаются" в них. Обычно вы не указываете новых имен полей, но если вы все-таки сделали это, вы должны делать это для каждого поля в представлении.

Когда делаете запрос представления, вы собственно, запрашиваете запрос. Основной способ для SQL – объединить предикаты двух запросов в один. Посмотрим еще раз на представление с именем Londonstaff:

```
CREATE VIEW Londonstaff  
AS SELECT *  
FROM Salespeople  
WHERE city = 'London';
```

Если мы выполняем следующий запрос в этом представлении

```
SELECT *  
FROM Londonstaff  
WHERE comm > .12;
```

он такой же, как если бы мы выполнили следующее в таблице Продавцов:

```
SELECT *  
FROM Salespeople  
WHERE city = 'London' AND comm > .12;
```

Но здесь появляется возможная проблема с представлением. Имеется возможность комбинации из двух полностью допустимых предикатов и получения предиката, который не будет работать. Например, предположим, что мы создаем (CREATE) следующее представление:

```
CREATE VIEW Ratingcount (rating, number)  
AS SELECT rating, COUNT (*)  
FROM Customers  
GROUP BY rating;
```

Это дает нам число заказчиков, которые мы имеем для каждого уровня оценки rating. Можно затем сделать запрос этого представления, чтобы выяснить, имеется ли какая-нибудь оценка, в настоящее время назначенная для трех заказчиков:

```
SELECT *  
FROM Ratingcount  
WHERE number = 3;
```

Посмотрим, что случится, если мы скомбинируем два предиката:

```
SELECT rating, COUNT (*)  
FROM Customers  
WHERE COUNT (*) = 3  
GROUP BY rating;
```

Это недопустимый запрос. Агрегатные функции, такие как COUNT, не могут использоваться в предикате. Правильным способом при формировании вышеупомянутого запроса, конечно же, будет следующий:


```
SELECT rating, COUNT (*)  
FROM Customers  
GROUP BY rating;  
HAVING COUNT (*) = 3;
```

Групповые представления могут стать хорошим способом обрабатывать полученную информацию непрерывно. Предположим, что каждый день надо следить за порядком номеров заказчиков, номерами продавцов, принимающих Заказы, номерами Заказов, средним от Заказов, и общей суммой приобретений в Заказах.

Вместо того, чтобы конструировать каждый раз сложный запрос, можно создать следующее представление:

```
CREATE VIEW Totalforday  
AS SELECT odate, COUNT(DISTINCT cnum), COUNT(DISTINCT snum),  
    COUNT(onum), AVG(amt), SUM(amt)  
FROM Orders  
GROUP BY odate;
```

Теперь можно увидеть всю эту информацию с помощью простого запроса:

```
SELECT *  
FROM Totalforday;
```

Представления не требуют, чтобы их вывод осуществлялся из одной базовой таблицы. Так как почти любой допустимый запрос SQL может быть использован в представлении, он может выводить информацию из любого числа базовых таблиц, или из других представлений.

Мы можем, например, создать представление, которое показывало бы Заказы продавца и заказчика по имени:

```
CREATE VIEW Nameorders  
AS SELECT onum, amt, a.snum, sname, cname  
    FROM Orders a, Customers b, Salespeople c  
    WHERE a.cnum = b.cnum AND a.snum = c.snum;
```

Теперь можно, например, выбрать все Заказы заказчика или продавца, или увидеть эту информацию для любого Заказа.

Например, чтобы увидеть все Заказы продавца Rifkin, надо сделать следующий запрос:

```
SELECT *  
FROM Nameorders  
WHERE sname = 'Rifkin';
```

Можно также объединять представления с другими таблицами, или базовыми таблицами или представлениями. Например, чтобы увидеть все заказы Axelrod и значения его комиссионных в каждом Заказе:

```
SELECT a.sname, cname, amt comm  
FROM Nameorders a, Salespeople b  
WHERE a.sname = 'Axelrod' AND b.snum = a.snum;
```

Представления могут также использовать и подзапросы, включая соотнесенные подзапросы. Например, компания предусматривает премию для тех продавцов, которые имеют заказчика с самой высокой суммой Заказа для любой указанной даты. Это можно проследить помощью представления:

```
CREATE VIEW Elitesalesforce  
AS SELECT b.odate, a.snum, a.sname,  
    FROM Salespeople a, Orders b  
    WHERE a.snum = b.snum AND b.amt = (SELECT MAX (amt)  
        FROM Orders c  
        WHERE c.odate = b.odate);
```

Если, с другой стороны, премия будет назначаться только продавцу, который имел самую высокую сумму Заказа за последние десять лет, вам необходимо будет проследить их в другом представлении, основанном на первом:

```
CREATE VIEW Bonus
AS SELECT DISTINCT snum, sname
FROM Elitesalesforce a
WHERE 10 <= (SELECT COUNT (*)
FROM Elitesalestorce b
WHERE a.snum = b.snum);
```

Извлечение из этой таблицы продавца, который будет получать премию, выполняется простым запросом:

```
SELECT *
FROM Bonus;
```

Удаление представлений

Синтаксис удаления представления из базы данных подобен синтаксису удаления базовых таблиц:

```
DROP VIEW <view name>
```

В этом нет необходимости, однако, сначала надо удалить все содержание, как это делается с базовой таблицей, потому что содержание представления не является созданным и сохраняется в течение определенной команды. Базовая таблица, из которой представление выводится, не эффективна, когда представление удалено.

Помните, вы должны являться владельцем представления, чтобы иметь возможность удалить его.

Модифицирование представления

Один из наиболее трудных и неоднозначных аспектов представлений – непосредственное их использование с командами модификации DML. Это является некоторым противоречием. Представление состоит из результатов запроса, и когда вы модифицируете представление, вы модифицируете набор результатов запроса. Но модификация не должна воздействовать на запрос; она должна воздействовать на значения в таблице, к которой был сделан запрос, и таким образом изменять вывод запроса.

Определение модифицируемости представления

Если команды модификации могут выполняться в представлении, представление как сообщалось, будет модифицируемым; в противном случае оно предназначено только для чтения при запросе. Не противореча этой терминологии, мы будем использовать выражение "модифицируемое представление" (updating a view), что означает возможность выполнения в представлении любой из трех команд модификации DML (INSERT, UPDATE и DELETE), которые могут изменять значения.

Как определяется, является ли представление модифицируемым? Критерии, по которым определяют, является ли представление модифицируемым или нет, в SQL, следующие:

- Оно должно выводиться в одну и только в одну базовую таблицу.
- Оно должно содержать первичный ключ этой таблицы (это технически не предписывается стандартом ANSI, но было бы неплохо придерживаться этого).
- Оно не должно иметь никаких полей, которые бы являлись агрегатными функциями.
- Оно не должно содержать DISTINCT в своем определении.
- Оно не должно использовать GROUP BY или HAVING в своем определении.
- Оно не должно использовать подзапросы (это — ANSI ограничение, которое не предписано для некоторых реализаций).
- Оно может быть использовано в другом представлении, но это представление должно также быть модифицируемым.
- Оно не должно использовать константы, строки, или выражения значений (например, comm*100) среди выбранных полей вывода.
- Для INSERT, оно должно содержать любые поля основной таблицы, которые имеют ограничение NOT NULL, если другое ограничение по умолчанию не определено.

Одно из этих ограничений то, что модифицируемые представления, фактически, подобны окнам в базовых таблицах. Они показывают кое-что, но не обязательно все, из содержимого

таблицы. Они могут ограничивать определенные строки (использованием предикатов), или специально именованные столбцы (с исключениями), но они представляют значения непосредственно и не выводят информацию с использованием составных функций и выражений.

Они также не сравнивают строки таблиц друг с другом (как в объединениях и подзапросах, или как с DISTINCT).

Различия между модифицируемыми представлениями и представлениями "только чтение" неслучайны.

Модифицируемые представления, в основном, используются точно так же, как и базовые таблицы. Фактически, пользователи не могут даже осознать, является ли объект, который они запрашивают, базовой таблицей или представлением. Это хороший механизм защиты для сокрытия частей таблицы, которые являются конфиденциальными или не относятся к потребностям данного пользователя.

Представления "только чтение", с другой стороны, позволяют получать и переформатировать данные более рационально. Они дают вам библиотеку сложных запросов, которые можно выполнить и повторить снова, сохраняя полученную ранее информацию. Кроме того, результаты этих запросов в таблицах, которые могут затем использоваться в запросах самостоятельно (например, в объединениях) имеют преимущество над просто выполнением запросов.

Имеются некоторые примеры модифицируемых представлений и представлений "только чтение":

```
CREATE VIEW Dateorders (odate, ocount)  
AS SELECT odate, COUNT (*)  
FROM Orders  
GROUP BY odate;
```

Это – представление "только чтение" из-за присутствия в нем агрегатной функции и GROUP BY.

```
CREATE VIEW Londoncust  
AS SELECT *  
FROM Customers  
WHERE city = 'London';
```

Это – представление модифицируемое.

```
CREATE VIEW SJsales (name, number, percentage)  
AS SELECT sname, snum, comm * 100  
FROM Salespeople  
WHERE city = 'SanJose';
```

Это – представление "только чтение" из-за выражения "comm * 100". При этом, однако, возможно переупорядочение и переименование полей. Некоторые программы будут позволять удаление в этом представлении или в Заказах столбцов snum и sname.

```
CREATE VIEW Salesonthird  
AS SELECT *  
FROM Salespeople  
WHERE snum IN (SELECT snum  
FROM Orders  
WHERE odate = 10/03/1990);
```

Это – представление "только чтение" в ANSI из-за присутствия в нем подзапроса. В некоторых программах, это может быть модифицируемым.

```
CREATE VIEW Someorders  
AS SELECT snum, onum, cnum  
FROM Orders  
WHERE odate IN (10/03/1990,10/05/1990);
```

Это – модифицируемое представление.

Проверка значений, помещаемых в представление

Другой вывод о модифицируемости представления тот, что вы можете вводить значения которые "проглатываются" (swallowed) в базовой таблице. Рассмотрим такое представление:

```
CREATE VIEW Highratings  
AS SELECT cnum, rating  
FROM Customers  
WHERE rating = 300;
```

Это — представление модифицируемое. Оно просто ограничивает ваш доступ к определенным строкам и столбцам в таблице. Предположим, что вы вставляете (INSERT) следующую строку:

```
INSERT INTO Highratings  
VALUES (2018, 200);
```

Это — допустимая команда INSERT в этом представлении. Строка будет вставлена, с помощью представления Highratings, в таблицу Заказчиков. Однако когда она появится там, она исчезнет из представления, поскольку значение оценки не равно 300. Это — обычная проблема.

Значение 200 может быть просто напечатано, но теперь строка находится уже в таблице Заказчиков, где вы не можете даже увидеть ее. Пользователь не сможет понять, почему введя строку, он не может ее увидеть, и будет неспособен при этом удалить ее.

Вы можете быть гарантированы от модификаций такого типа с помощью включения WITH CHECK OPTION (с опцией проверки) в определение представления. Мы можем использовать WITH CHECK OPTION в определении представления Highratmgs.

```
CREATE VIEW Highratings  
AS SELECT cnum, rating  
FROM Customers  
WHERE rating = 300  
WITH CHECK OPTION;
```

Вышеупомянутая вставка будет отклонена.

WITH CHECK OPTION — производит действие все_или_ничего (all-or-nothing). Вы помещаете его в определение представления, а не в команду DML, так что или все команды модификации в представлении будут проверяться, или ни одна не будет проверена. В общем, вы должны использовать эту опцию, если у вас нет причины разрешать представлению помещать в таблицу значения, которые он сам не может содержать.

Предикаты и исключенные поля

Похожая проблема, которую вы должны знать, включает в себя вставку строк в представление с предикатом, базирующемся не на всех полях таблицы. Например, может показаться разумным создать Londonstaff подобно этому:

```
CREATE VIEW Londonstaff  
AS SELECT snum, sname, comm  
FROM Salespeople  
WHERE city = 'London';
```

В конце концов, зачем включать значение city, если все значения city будут одинаковыми.

А как будет выглядеть картинка, получаемая всякий раз, когда мы пробуем вставить строку? Так как мы не можем указать значение city как значение по умолчанию, этим значением, вероятно, будет NULL, и оно будет введено в поле city. Так как в этом случае поле city не будет равняться значению London, вставляемая строка будет исключена из представления. Это будет верным для любой строки, которую вы попытаетесь вставить в просмотр Londonstaff. Все они должны быть введены с помощью представления Londonstaff в таблицу Продавцов, и затем исключены из самого представления (если определением по умолчанию был не London, то это особый случай). Пользователь не сможет вводить строки в это представление, хотя все еще неизвестно, может ли он вводить строки в базовую таблицу. Даже если мы добавим WITH CHECK OPTION в определение представления

```
CREATE VIEW Londonstate  
AS SELECT snum, sname, comm  
FROM Salespeople  
WHERE city = 'London'  
WITH CHECK OPTION;
```

проблема не обязательно будет решена. В результате этого мы получим представление, которое мы могли бы модифицировать или из которого мы могли бы удалять, но не вставлять в него.

Даже если это не всегда может обеспечить Вас полезной информацией, полезно включать в ваше представление все поля, на которые имеется ссылка в предикате. Если вы не хотите видеть эти поля в вашем выводе, вы всегда сможете исключить их из запроса в представлении, в противоположность запросу внутри представления. Другими словами, вы могли бы определить представление Londonstaff подобно этому:

```
CREATE VIEW Londonstaff  
AS SELECT *  
FROM Salespeople  
WHERE city = 'London'  
WITH CHECK OPTION;
```

Эта команда заполнит представление одинаковыми значениями в поле city, которые вы можете просто исключить из вывода с помощью запроса, в котором указаны только те поля, которые вы хотите видеть:

```
SELECT snum, sname, comm  
FROM Londonstaff;
```

Проверка представлений, которые базируются на других представлениях

Еще одно надо упомянуть относительно предложения WITH CHECK OPTION в ANSI: оно не делает каскадированного изменения: Оно применяется только в представлениях, в которых оно определено, но не в представлениях основанных на этом представлении. Например, в предыдущем примере

```
CREATE VIEW Highratings  
AS SELECT cnum, rating  
FROM Customers  
WHERE rating = 300  
WITH CHECK OPTION;
```

попытка вставить или модифицировать значение оценки не равное 300 потерпит неудачу. Однако, мы можем создать второе представление (с идентичным содержанием) основанное на первом:

```
CREATE VIEW Myratings  
AS SELECT *  
FROM Highratings;
```

Теперь мы можем модифицировать оценки, не равные 300:

```
UPDATE Myratings  
SET rating = 200  
WHERE cnum = 2004;
```

Эта команда, выполняемая так, как если бы она выполнялась как первое представление, будет допустима. Предложение WITH CHECK OPTION просто гарантирует, что любая модификация в представлении произведет значения, которые удовлетворяют предикату этого представления. Модификация других представлений, базирующихся на первом текущем, является все еще допустимой, если эти представления не защищены предложениями WITH CHECK OPTION внутри этих представлений. Даже если такие предложения установлены, они проверяют только те предикаты представлений, в которых они содержатся. Так, например, даже если представление Myratings создавалось следующим образом

```
CREATE VIEW Myratings  
AS SELECT *  
FROM Highratings  
WITH CHECK OPTION;
```

проблема не будет решена. Предложение WITH CHECK OPTION будет исследовать только предикат представления Myratings. Пока у Myratings, фактически, не имеется никакого предиката, WITH CHECK OPTION ничего не будет делать. Если используется предикат, то он будет проверяться всякий раз, когда представление Myratings будет модифицироваться, но предикат Highratings все равно будет проигнорирован.